Module 3

Attention Mechanisms and Transformer Architecture

Selina Ochukut

Module Overview.

This module introduces *attention*, the mechanism that underlies nearly all modern large-scale sequence models, and the *Transformer* architecture built entirely around it. You will begin with the limitations of recurrent sequence models that motivated attention, derive self-attention and multi-head attention from first principles, and then assemble the full encoder–decoder Transformer architecture layer by layer. You will end with a survey of specialized attention variants (sparse, linear, local, relative-positional, and hardware-aware attention) that make Transformers practical at very long sequence lengths and very large scale.

Expected Learning Outcomes

1. Describe the mathematical role of queries, keys, values, and the scaling factor in the attention mechanism.
2. **Explain** the structure and function of each component of a Transformer block.
3. **Analyze** how information flows through a Transformer architecture and compare the computational and memory characteristics of standard self-attention with efficient attention mechanisms.
4. **Evaluate** the trade-offs of modern efficient-attention variants.

Self-Attention and Multi-Head Attention

Motivation: The Limits of Recurrence

Before Transformers, sequence modeling (machine translation, language modeling, speech recognition) was dominated by **Recurrent Neural Networks (RNNs)** and their gated variants (LSTM, GRU). An RNN processes a sequence $x_1, x_2, \ldots, x_n$ one element at a time, maintaining a hidden state $h_t = f(h_{t-1}, x_t)$ that summarizes everything seen so far.

This design has two fundamental limitations:

- **Sequential computation:** h_t cannot be computed until h_{t-1} is known. This strictly sequential dependency prevents parallelization across the time dimension during training, making RNNs slow on modern hardware (GPUs/TPUs), which are optimized for large parallel matrix operations.
- **Long-range dependency bottleneck:** Information about x_1 must travel through every intermediate hidden state to influence a prediction at time n. Even with gating mechanisms designed to mitigate this (LSTM/GRU), in practice the effective "memory" of an RNN degrades over long distances – a problem often described in terms of vanishing gradients during backpropagation through time.

Early attention mechanisms (Bahdanau et al., 2014) were introduced as an add-on to RNN-based encoder-decoder models for machine translation, allowing the decoder to "look back" at *all* encoder hidden states at every decoding step, rather than relying solely on a single fixed-size context vector. The key insight that followed (Vaswani et al., 2017, *"Attention Is All You Need"*) was that attention alone – with no recurrence at all – is sufficient to build state-of-the-art sequence models, and additionally solves both limitations above: attention can be computed for all sequence positions in parallel, and every position can directly attend to every other position in a single step, regardless of distance.

Key Idea

Self-attention replaces the RNN's sequential, one-step-at-a-time information flow with a mechanism where *every* position in a sequence can directly and simultaneously gather information from *every other* position, weighted by a learned notion of relevance. This trades $O(n)$ sequential steps for $O(1)$ sequential steps (fully parallel), at the cost of $O(n^2)$ computation per layer.

Queries, Keys, and Values

Self-attention is best understood through an information-retrieval analogy. Suppose we have a database of items, each stored as a (key, value) pair. To retrieve information, we issue a *query*, compare it against every key to obtain a relevance score, and then return a weighted combination of the values, weighted by these scores.

Definition

For an input sequence of n token embeddings, each of dimension d_{model}, arranged as a matrix $X \in \mathbb{R}^{n \times d_{model}}$, self-attention first projects X into three learned representations using weight matrices $W^Q, W^K \in \mathbb{R}^{d_{model} \times d_k}$ and $W^V \in \mathbb{R}^{d_{model} \times d_v}$:

$$Q = XW^Q, \qquad K = XW^K, \qquad V = XW^V$$

- Q (**Query**): "what am I looking for?", one row per position.
- K (**Key**): "what do I contain?", one row per position – compared against queries to compute relevance.
- V (**Value**): "what information do I actually pass on?", one row per position – combined according to the relevance weights.

Key Idea

Crucially, Q, K, and V are all computed from the *same* input X in self-attention (hence "self"): every position simultaneously plays the role of a query (looking for relevant context), a key (being looked up by others), and a value (contributing content). This differs from *cross-attention*, discussed in Section 2, where queries come from one sequence and keys/values come from a different sequence.

Scaled Dot-Product Attention

Definition. *Given* $Q \in \mathbb{R}^{n \times d_k}$, $K \in \mathbb{R}^{n \times d_k}$, $V \in \mathbb{R}^{n \times d_v}$, ***scaled dot-product attention*** *is defined as:*

$$Attention(Q, K, V) = softmax\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Let us unpack this expression step by step:

1. $QK^\top \in \mathbb{R}^{n\times n}$**:** Each entry $(QK^\top)_{ij} = q_i \cdot k_j$ is the dot product between the query at position i and the key at position j, measuring how relevant position j is to position i. This produces an $n \times n$ matrix of raw relevance scores for every pair of positions.

2. **Scaling by** $\frac{1}{\sqrt{d_k}}$**:** Without scaling, as d_k grows, dot products tend to grow large in magnitude (since they sum d_k terms), pushing the softmax function into regions with extremely small gradients (saturation). Dividing by $\sqrt{d_k}$ counteracts this: assuming query and key components are independent random variables with mean 0 and variance 1, the dot product $q \cdot k$ has variance d_k, so dividing by $\sqrt{d_k}$ renormalizes it back to unit variance.

3. **softmax**$(\cdot)$ **(row-wise):** Converts each row of raw scores into a probability distribution over the n positions – the **attention weights** – so that for each query position i, the weights $\alpha_{i1}, \ldots, \alpha_{in}$ sum to 1 and indicate how much "attention" position i pays to each position j.

4. **Multiplying by** V**:** The output for position i is $\sum_j \alpha_{ij} v_j$ – a weighted average of all value vectors, weighted by relevance. The output has the same number of rows (n) as the input, so self-attention can be stacked or combined with other layers of the same shape.

Example 1. *Consider a toy sequence of 3 tokens with* $d_k = d_v = 2$*. Suppose for query position* 1*, the raw dot products (before scaling) with keys* $1, 2, 3$ *are* $[4, 0, 2]$*, and* $d_k = 4$ *so we scale by* $\frac{1}{\sqrt{4}} = 0.5$*, giving scaled scores* $[2, 0, 1]$*. Applying softmax:*

$$softmax([2, 0, 1]) \approx [0.659, 0.090, 0.245]$$

If $v_1 = [1, 0]$*,* $v_2 = [0, 1]$*,* $v_3 = [1, 1]$*, the output for position 1 is:*

$$0.659[1, 0] + 0.090[0, 1] + 0.245[1, 1] \approx [0.904, 0.335]$$

Position 1's output is therefore dominated by v_1 *(since it received the highest attention weight), with smaller contributions from* v_2 *and* v_3*.*

Multi-Head Attention

A single attention computation forces the model to average all types of relevant relationships (e.g., syntactic agreement, coreference, positional proximity) into one set of attention weights per pair of positions. **Multi-head attention** instead runs several independent attention computations ("heads")

in parallel, each operating in a lower-dimensional projected subspace, allowing different heads to specialize in different kinds of relationships.

Definition

Given h heads, each with its own learned projections $W_i^Q, W_i^K \in \mathbb{R}^{d_{model} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{model} \times d_v}$ (typically $d_k = d_v = d_{model}/h$):

$$\text{head}_i = \text{Attention}(XW_i^Q, XW_i^K, XW_i^V), \qquad i = 1, \ldots, h$$

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W^O$$

where $W^O \in \mathbb{R}^{hd_v \times d_{model}}$ is a final learned output projection that mixes information across heads back into a single d_{model}-dimensional representation per position.

Key Idea

Splitting a d_{model}-dimensional representation into h heads of dimension d_{model}/h each keeps the *total* computational cost of multi-head attention roughly the same as a single full-dimensional attention computation, while allowing the model to attend to information from different representation subspaces at different positions simultaneously. Empirically, different heads are often found to specialize – e.g., one head might learn to track subject-verb agreement, another might attend primarily to the immediately preceding token, another might track long-range coreference.

Positional Encoding

Notice that scaled dot-product attention, as defined above, is **permutation equivariant**: if we shuffle the rows (positions) of X, the output rows are shuffled correspondingly, but each output value is otherwise unchanged. Attention alone has no notion of *order* – "the dog bit the man" and "the man bit the dog" would produce identical (permuted) self-attention outputs at the level of the raw mechanism, since it only depends on content, not position. Since word order is obviously critical to meaning, the Transformer must inject positional information explicitly.

Definition

Sinusoidal positional encoding adds a fixed, non-learned vector $PE(pos, \cdot)$ to each input embedding based on its position pos in the sequence:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$
$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

where i indexes the dimension pair. The final input to the first Transformer layer is $X + PE$.

Key Idea

Sinusoidal encodings were chosen (rather than, say, simply using learned positional embeddings) because they allow the model to extrapolate to sequence lengths not seen during training, and because for any fixed offset k, $PE(pos+k)$ can be expressed as a linear function of $PE(pos)$, which the original authors hypothesized would make it easy for the model to learn to attend by *relative* position. In practice, many modern Transformers instead use **learned** positional embeddings, or the **relative** positional encoding schemes discussed in Section 3.

Masking

Two distinct kinds of masks are used in Transformer attention:

- **Padding mask:** Sequences in a batch are padded to a common length; the padding mask sets attention scores involving padding tokens to $-\infty$ before the softmax, ensuring padding contributes zero weight and is never attended to.
- **Causal (look-ahead) mask:** Used in autoregressive decoding (e.g., language modeling, the Transformer decoder), this mask prevents position i from attending to any position $j > i$, by setting those scores to $-\infty$ before the softmax. This preserves the autoregressive property: predictions for position i may only depend on positions $1, \ldots, i-1$ (or i, depending on convention), never on "future" tokens, which is essential both for valid language generation and to prevent the model from trivially "cheating" during training by looking at the answer.

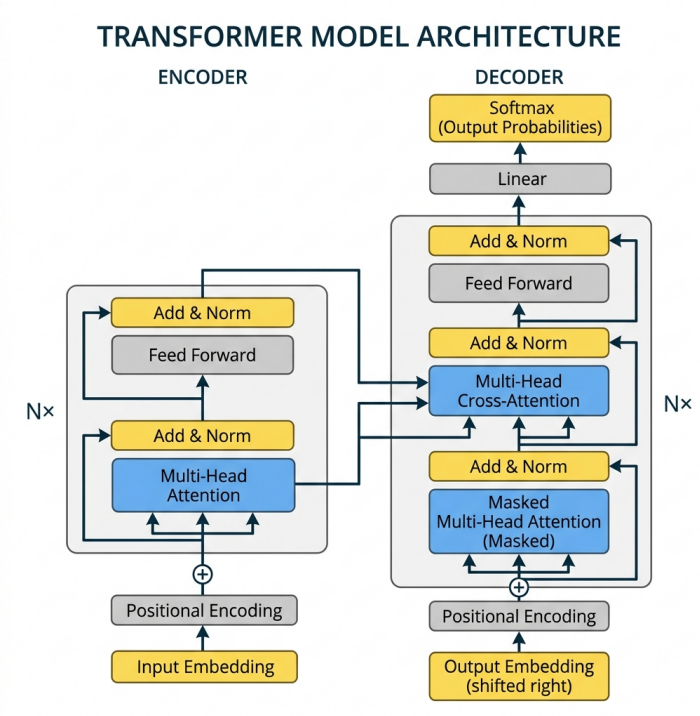
Transformer Architecture

Video Visit the URL below to view a video:

https://www.youtube.com/embed/TQQlZhbC5ps

The Transformer (Vaswani et al., 2017) is built entirely from the attention mechanism introduced above, combined with position-wise feed-forward networks, residual connections, and layer normalization, arranged into a stack of **encoder** and **decoder** blocks. The original architecture was designed for sequence-to-sequence tasks (machine translation), and its encoder-only and decoder-only variants now underlie the majority of modern large language models.

Overall Architecture



- The **encoder** (left) maps an input sequence into a sequence of continuous representations, using N identical stacked layers, each containing a multi-head self-attention sublayer and a feed-forward sublayer.
- The **decoder** (right) generates the output sequence autoregressively (one token at a time), using N identical stacked layers, each containing a *masked* self-attention sublayer (so it cannot see future output tokens), a **cross-attention** sublayer (attending over the encoder's output), and a feed-forward sublayer.
- Every sublayer is wrapped with a **residual connection** followed by **layer normalization**: $\text{LayerNorm}(x + \text{Sublayer}(x))$.

Position-Wise Feed-Forward Network

Each Transformer layer (encoder and decoder) contains a feed-forward network applied *independently and identically* to each position:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

This is a two-layer MLP with a ReLU (or GELU, in many modern variants) nonlinearity, typically expanding the dimensionality substantially in the hidden layer (e.g., from $d_{model} = 512$ to $d_{ff} = 2048$) before projecting back down to d_{model}.

Key Idea

While self-attention mixes information *across* positions (each position's output depends on all other positions), the feed-forward network processes each position *independently*, applying the same learned transformation to every position's representation. Attention layers are therefore responsible for all cross-token interaction, while feed-forward layers add per-token representational capacity and nonlinearity. The same two weight matrices W_1, W_2 are reused (shared) across all positions within a layer, though different layers have their own separate feed-forward parameters.

Residual Connections and Layer Normalization

Definition

A **residual (skip) connection** adds a sublayer's input directly to its output: $x + \text{Sublayer}(x)$, rather than replacing x entirely. **Layer normalization** then normalizes the resulting sum across the feature dimension (per position, per example) to have zero mean and unit variance, followed by a learned scale and shift:

$$\text{LayerNorm}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

where μ, σ^2 are computed across the feature dimension for each position independently.

Key Idea

Residual connections address the vanishing-gradient / difficult-optimization problem in very deep networks: the gradient can flow directly through the identity path $x \to x$, bypassing the sublayer entirely if needed, which makes stacking many layers ($N = 6$ in the original Transformer, but far more in modern large models) trainable in practice. Layer normalization (as opposed to batch normalization, common in CNNs) normalizes across the *feature* dimension independently for each sequence position, which is well suited to variable-length sequences and avoids any dependence on batch statistics, making it stable even with small or variable batch sizes.

Two conventions exist for where normalization is applied relative to the sublayer: **Post-LN** (as in the original paper: $\text{LayerNorm}(x + \text{Sublayer}(x))$) and **Pre-LN** ($x + \text{Sublayer}(\text{LayerNorm}(x))$), used in many later models (e.g., GPT-2 and beyond) because it tends to produce more stable gradients and training dynamics, particularly for very deep stacks.

Cross-Attention

The decoder's second attention sublayer is **cross-attention** (also called *encoder-decoder attention*): the queries Q come from the decoder's own (masked self-attention) representations, while the keys K and values V come from the *encoder's final output*.

Key Idea

Cross-attention is what allows the decoder to condition its generation on the full input sequence: at each decoding step, the decoder asks (via its query) "given what I've generated so far, which parts of the input sequence are most relevant right now?" and retrieves a weighted combination of encoder representations accordingly. This is the direct, more powerful descendant of the original Bahdanau-style attention used in RNN encoder-decoder models, and is structurally identical to self-attention except that K, V come from a different sequence than Q.

Encoder-Only, Decoder-Only, and Encoder-Decoder Variants

The full encoder-decoder Transformer is well suited to sequence-to-sequence tasks (translation, summarization). Two influential simplifications emerged for other use cases:

- **Encoder-only** (e.g., BERT): Uses only the encoder stack, with *no* causal masking, so every position can attend to every other position (including "future" ones) – suited to tasks requiring a rich bidirectional representation of a fixed input, such as classification, span extraction, or token tagging, but not well suited to autoregressive generation.
- **Decoder-only** (e.g., GPT family): Uses only the (causally masked) decoder self-attention sublayer – no cross-attention, since there is no separate encoder – trained to predict the next token given all previous tokens. This is the dominant architecture for modern large generative language models.
- **Encoder-decoder** (e.g., the original Transformer, T5, BART): Retains both stacks, well suited to tasks with a clear separation between an input sequence to be understood and an output sequence to be generated.

Computational Complexity

Layer type	**Complexity per layer**	**Sequential ops**	**Max path length**
Self-attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent (RNN)	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k n)$

where n is sequence length, d is representation dimension, and k is convolution kernel width.

Key Idea

Self-attention's $O(1)$ sequential operations and $O(1)$ maximum path length between any two positions (any position can attend directly to any other in a single layer) are its central computational advantages over recurrence. Its main disadvantage is the $O(n^2)$ dependence on sequence length, which becomes the dominant bottleneck for very long sequences (thousands to millions of tokens) – directly motivating the specialized, sub-quadratic attention variants discussed in Section 3.

Training Considerations

- **Teacher forcing:** During training, the decoder is fed the ground-truth previous tokens (rather than its own predictions) as input at each step, which – combined with the causal mask – allows the entire output sequence's loss to be computed in a single parallel forward pass, rather than one token at a time.
- **Label smoothing:** Instead of training against a one-hot target distribution, a small probability mass (ϵ, e.g., 0.1) is redistributed from the correct token to all other tokens, which the original authors found improved BLEU score and calibration, at a small cost to perplexity.
- **Learning rate warm-up:** The original Transformer used a learning rate schedule that linearly increases for a fixed number of "warm-up" steps and then decays proportionally to the inverse square root of the step number, which was found to stabilize early training of the deep, residual, attention-based stack.

Specialized Attention Variations

The $O(n^2)$ complexity of standard self-attention, along with practical deployment constraints (memory bandwidth, inference latency, extrapolation to long contexts), has motivated a large family of specialized attention mechanisms. This section surveys the most influential variants, organized by the problem each one primarily addresses.

Sparse Attention

Motivation: If most of the useful signal in an attention matrix is concentrated in a small subset of position pairs, computing and storing the *full* $n \times n$ matrix is wasteful. Sparse attention mechanisms restrict each position to attend to only a fixed, structured subset of other positions.

- **Local (sliding-window) attention:** Each position attends only to a fixed-size window of nearby positions (e.g., the 512 preceding and following tokens), reducing complexity to $O(n \cdot w)$ where w is the window size. This captures short-range dependencies efficiently but cannot directly model very long-range interactions within a single layer.
- **Global + local attention (Longformer):** Combines local windowed attention (for most tokens) with a small number of designated **global tokens** (e.g., a `[CLS]` token, or task-specific tokens) that attend to, and are attended to by, *every* position – providing a mechanism for long-range information to still propagate through the global tokens, while keeping most of the computation local.
- **Random + local + global attention (BigBird):** Extends this idea further by additionally allowing each position to attend to a small number of *random* other positions, which the authors show (via a graph-theoretic connectivity argument) allows information to propagate across the full sequence in a small number of layers while keeping overall attention sparse and near-linear in n.

Key Idea

Sparse attention patterns trade the ability to directly attend to every position (in a single layer) for reduced computation and memory. The design challenge is choosing a sparsity pattern that still allows information to propagate across the whole sequence within a reasonable number of layers – purely local attention alone cannot do this,

which is why global tokens or random connections are typically added alongside it.

Linear Attention

Motivation: Rather than sparsifying *which* pairs are computed, linear attention methods reformulate the attention computation itself to avoid ever materializing the full $n \times n$ matrix, achieving $O(n)$ complexity in sequence length.

Key Idea

The softmax in standard attention, $\text{softmax}(QK^\top)V$, cannot generally be reordered to avoid the $n \times n$ intermediate, because softmax is applied to the full row of scores. Linear attention methods replace $\text{softmax}(QK^\top)$ with a similarity function that can be expressed as $\phi(Q)\phi(K)^\top$ for some (possibly approximate) feature map ϕ. Since matrix multiplication is associative, this allows computing $\phi(K)^\top V$ *first* (an $O(n)$ operation producing a fixed-size $d \times d$ matrix), and then $\phi(Q)\big(\phi(K)^\top V\big)$ (also $O(n)$), avoiding the $n \times n$ intermediate entirely.

- **Linear Transformers (Katharopoulos et al.):** Use a simple feature map such as $\phi(x) = \text{elu}(x) + 1$ to approximate the effect of the softmax kernel, achieving exact $O(n)$ attention (at the cost of some approximation of the true softmax behavior).
- **Performer (FAVOR+):** Uses random feature approximations (based on positive orthogonal random features) to approximate the softmax kernel unbiasedly, with theoretical guarantees on approximation quality, again yielding $O(n)$ complexity.

Trade-off: Linear attention methods achieve favorable asymptotic complexity, but in practice can underperform standard softmax attention in quality, particularly on tasks that benefit from softmax's sharply peaked (near-one-hot) attention distributions; they are most beneficial when sequence length n is very large relative to d.

Multi-Query and Grouped-Query Attention

Motivation: During autoregressive decoding, keys and values for all previous tokens are cached (the **KV cache**) to avoid recomputing them at every

generation step. For large models generating long sequences, the memory required to store this cache (proportional to the number of attention heads) becomes a major bottleneck for inference throughput and latency, independent of the $O(n^2)$ compute cost.

- **Multi-Query Attention (MQA):** All query heads share a *single* set of key and value projections (instead of each head having its own W_i^K, W_i^V), drastically shrinking the KV cache (by a factor of h, the number of heads) at the cost of some model quality.
- **Grouped-Query Attention (GQA):** An interpolation between standard multi-head attention (each head has its own K, V) and MQA (all heads share one K, V): heads are partitioned into a smaller number of groups, with all heads in a group sharing one set of K, V projections. GQA is used in several modern large language models as a practical compromise, recovering most of the inference speed benefit of MQA with a smaller quality loss.

Key Idea

MQA and GQA address an *inference-time memory bandwidth* bottleneck rather than the training-time $O(n^2)$ compute bottleneck targeted by sparse/linear attention – illustrating that "efficient attention" research addresses several distinct practical constraints (compute, memory, latency), not a single unified problem.

Relative Positional Encodings

Motivation: Absolute sinusoidal or learned positional encodings (Section 1.5) are added once, at the input, and their influence on attention scores is indirect. Several architectures instead inject positional information *directly into the attention computation itself*, as a function of the relative distance between the query and key positions, which often generalizes better to sequence lengths not seen during training.

- **Transformer-XL relative positional encoding:** Reformulates the attention score to depend on the relative offset $i - j$ between query position i and key position j, rather than their absolute positions, and introduces a **segment-level recurrence** mechanism that caches representations from previous segments, allowing context to extend beyond a single fixed-length training window.

- **Rotary Position Embedding (RoPE):** Encodes absolute position by *rotating* query and key vectors (in pairs of dimensions) by an angle proportional to their position, such that the dot product between a rotated query at position i and a rotated key at position j depends only on their *relative* offset $i - j$, combining the simplicity of absolute encodings with the desirable relative-distance properties of relative encodings. RoPE is widely used in modern large language models (e.g., the LLaMA family, GPT-NeoX).
- **ALiBi (Attention with Linear Biases):** Adds no positional embeddings to the input at all; instead, it directly biases attention scores by subtracting a penalty proportional to the distance $|i - j|$ between query and key positions (scaled by a head-specific slope), which the authors show enables strong extrapolation to sequence lengths considerably longer than those seen during training.

Key Idea

Relative positional schemes share a common motivation: what typically matters for language is the *distance* between two tokens, not their absolute position in the document. Encoding this relative information directly (rather than hoping the model learns to derive it from absolute encodings) tends to improve both sample efficiency and the ability to generalize to sequence lengths longer than those seen during training.

FlashAttention (Hardware-Aware Attention)

Motivation: On modern GPUs, the bottleneck in computing standard attention is often not the raw number of floating-point operations, but the cost of repeatedly reading and writing the large $n \times n$ intermediate score matrix to and from slower GPU high-bandwidth memory (HBM), rather than keeping computation within the much faster on-chip SRAM.

Definition

FlashAttention computes an *exact* (mathematically identical) scaled dot-product attention output, but restructures the computation using **tiling** (processing Q, K, V in blocks that fit in fast on-chip memory) and **online softmax** (incrementally updating softmax normalization statistics block-by-block, without ever materializing the

full $n \times n$ score matrix in slow memory), and recomputation during the backward pass instead of storing the full attention matrix.

Key Idea

Unlike sparse or linear attention, FlashAttention introduces *no approximation whatsoever* – it computes exactly the same output as standard softmax attention. Its speedup comes purely from being **IO-aware**: minimizing the number of slow memory reads/writes by restructuring the order of computation to exploit the GPU memory hierarchy. This makes it a purely systems-level optimization, fully compatible with any model architecture that uses standard attention, and it has become the default attention implementation in most modern training and inference frameworks.

Comparative Summary

Variant	**Problem addressed**	**Mechanism**	**Exact/Approx.?**
Sparse	$O(n^2)$ compute/memory for long sequences	Fixed sparsity pattern	Approximate
Linear	$O(n^2)$ compute/memory	Kernel feature maps	Approximate
Multi/Grouped	Inference-time KV cache	Shared K, V projections	Approximate
RoPE/ALiBi	Long-range generalization	Relative positional info	Exact
FlashAttention	GPU memory bandwidth	Tiling + online softmax	**Exact**

REFERENCES

- Prince, Simon JD. Understanding deep learning. MIT press, 2023.
- Goodfellow, I., Bengio, Y., Courville, A., & Bengio, Y. (2016). Deep learning (Vol. 1, No. 2, pp. 1-800). Cambridge: MIT press. https:

//www.deeplearningbook.org/

- Bishop, Christopher M., and Hugh Bishop. Deep learning: Foundations and concepts. Vol. 1. Cham, Switzerland: Springer, 2024.
- Torralba, A., Isola, P., & Freeman, W. T. (2024). Foundations of computer vision. MIT Press.